RÉSUMÉ : SERVICES WEB SOAP

Partie 1/3 — Services Web étendus et rappels XML

Badia Bouhdid (github.com/badi3a)

Architecture Orientée Services
Chapitre 2 · 2026-2027

1 MISE EN SITUATION

Une application **Java** doit récupérer les informations d'un étudiant auprès d'une application **PHP** distante. Elles ne partagent ni langage, ni mémoire, ni structure de données.



Question : comment se comprendre malgré ces différences ? **Réponse :** des **standards communs fondés sur XML** (SOAP, WSDL, UDDI).

2 LES STANDARDS D'UN SERVICE WEB ÉTENDU

WSDL	SOAP	UDDI
Web Services Description Language	historiquement Simple Object Access Protocol	Universal Description, Discovery and Integration
Contrat de description	**Protocole d'échange**	**Annuaire (historique)**
Quelles opérations ? quels types ? quelle adresse ?	Comment formuler la requête et la réponse ?	Où trouver ce service ?

3 FONCTIONNEMENT D'UN SERVICE WEB ÉTENDU

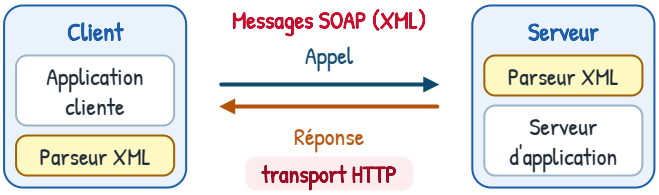


Image à retenir : HTTP achemine le message, comme une enveloppe postale ; SOAP définit ce qui est écrit à l'intérieur. De chaque côté, un parseur XML lit et produit les messages.

4 RAPPELS XML : DOCUMENT BIEN FORMÉ

```
<etudiant>
 <nom>Ben Ali</nom>
 <prenom>Sami</prenom>
 <age>19</age>
</etudiant>
```

- Toute balise ouverte est fermée
- Une seule balise racine
- Imbrication stricte, sans chevauchement
- Sensible à la casse : <Nom> ≠ <nom>

Bien formé ≠ valide. **Bien formé** : respecte ces règles de syntaxe. **Valide** : bien formé **et** conforme à un schéma (XSD ou DTD).

5 LES NAMESPACES XML

Problème : un message SOAP mélange le vocabulaire du **protocole** (Envelope, Header, Body) et le vocabulaire **métier**. Deux balises Body → **collision de noms.**

Solution : préfixer chaque balise par un namespace.

```
<soap:Envelope xmlns:soap="…/soap-envelope"
    xmlns:etu="…/gestionEtudiant">
  <soap:Body>
    <etu:getEtudiant>
      <etu:id>12</etu:id>
    </etu:getEtudiant>
  </soap:Body>
</soap:Envelope>
```

`xmlns:soap="…"` déclare un préfixe lié à une URI unique ; `soap:Body` devient une balise non ambiguë.

À retenir : le namespace est la **carte d'identité** d'une balise : son nom et à qui elle appartient. L'URI n'est jamais visitée par le parseur : c'est seulement un identifiant unique.

6 XSD : TYPER LE CONTENU XML

Comment dire que **id** est un entier et **moyenne** un décimal ? Avec un **schéma XSD** (XML Schema).

```
<xsd:complexType name="Etudiant">
 <xsd:sequence>
 <xsd:element name="id" type="xsd:int"/>
 <xsd:element name="nom"
      type="xsd:string"/>
 <xsd:element name="moyenne"
      type="xsd:decimal"/>
 </xsd:sequence>
</xsd:complexType>
```

Le schéma permet de **valider** les messages côté client et côté serveur : une chaîne envoyée là où un entier est attendu est détectée comme erreur.

! À NE PAS CONFONDRE

Bien formé (syntaxe correcte) ≠ **valide** (conforme à un schéma XSD).

SOAP (format du message) ≠ **HTTP** (transport qui l'achemine).

WSDL décrit le service, **SOAP** transporte l'appel, **UDDI** aide à le trouver.

RÉSUMÉ : SERVICES WEB SOAP

Partie 2/3 — Le message SOAP

Badia Bouhdid (github.com/badi3a)

Architecture Orientée Services
Chapitre 2 · 2026-2027

7 STRUCTURE D'UN MESSAGE SOAP

Message SOAP (porté par une requête HTTP)

En-têtes HTTP

SOAP Envelope

SOAP Header

En-têtes (optionnel)

SOAP Body

Appel de méthode & données

Fault (en cas d'erreur)

Envelope	Élément **racine**, **obligatoire**.
Header	**Optionnel** : sécurité, session, transaction…
Body	**Obligatoire** : la requête ou la réponse réelle.
Fault	Élément **du Body**, utilisé en cas d'erreur.

Le namespace de l'enveloppe indique la version :
SOAP 1.1 → `http://schemas.xmlsoap.org/soap/envelope/`
SOAP 1.2 → `http://www.w3.org/2003/05/soap-envelope`

À retenir : SOAP ne dépend pas d'un transport particulier ; HTTP est simplement le plus utilisé.

8 REQUÊTE ET RÉPONSE SOAP

Requête — opération `getEtudiant(12)`

```
<soap:Envelope xmlns:soap="…"
    xmlns:etu="…">
  <soap:Header/>
  <soap:Body>
    <etu:getEtudiantRequest>
      <etu:id>12</etu:id>
    </etu:getEtudiantRequest>
  </soap:Body>
</soap:Envelope>
```

Réponse — contenu du Body

```
<soap:Body>
  <etu:getEtudiantResponse>
    <etu:nom>Ben Ali</etu:nom>
    <etu:moyenne>14.5</etu:moyenne>
  </etu:getEtudiantResponse>
</soap:Body>
```

soap: = vocabulaire du protocole · **etu:** = vocabulaire métier.

Pourquoi le Header est-il vide ? Il est optionnel ; le Body, lui, porte toujours la requête ou la réponse.

9 GESTION DES ERREURS : SOAP FAULT

En cas d'erreur, le Body contient un élément **Fault** au lieu de la réponse.

```
<soap:Body>
  <soap:Fault>
    <faultcode>soap:Client</faultcode>
    <faultstring>
      Etudiant id=99 introuvable
    </faultstring>
  </soap:Fault>
</soap:Body>
```

faultcode : code de l'erreur, lu par le programme (ex. *Client* : la requête est en cause ; *Server* : le serveur a échoué).

faultstring : explication lisible par un humain.

Pourquoi dans le Body et non dans le Header ? Le Fault concerne directement le résultat de l'opération demandée ; ce n'est pas une métadonnée annexe comme les infos de session.

Ces noms sont ceux de SOAP 1.1. En SOAP 1.2 : **Code** et **Reason**, avec les codes *Sender* et *Receiver*.

10 TYPE SIMPLE VS TYPE COMPLEXE DANS LE BODY

getEtudiant(int id) → type simple

Un seul paramètre, **plat**, directement dans le Body.

```
<etu:getEtudiantRequest>
  <etu:id>12</etu:id>
</etu:getEtudiantRequest>
```

ajouterEtudiant(Etudiant e) → type complexe

Etudiant regroupe nom, moyenne et dateNaissance : les champs sont **imbriqués**.

```
<etu:ajouterEtudiantRequest>
  <etu:nouvelEtudiant>
    <etu:nom>Toto</etu:nom>
    <etu:moyenne>13.2</etu:moyenne>
    <etu:dateNaissance>2004-03-10</etu:dateNaissance>
  </etu:nouvelEtudiant>
</etu:ajouterEtudiantRequest>
```

Transition : pour écrire ces messages, le client doit connaître le nom de l'opération, le namespace, la structure attendue et l'adresse du service. C'est exactement ce que décrit le **contrat WSDL**.

RÉSUMÉ : SERVICES WEB SOAP

Partie 3/3 — Contrat WSDL, développement et UDDI

Badia Bouhdid (github.com/badi3a)

Architecture Orientée Services
Chapitre 2 · 2026-2027

11 LE CONTRAT WSDL

WSDL (Web Services Description Language) : langage basé sur XML qui décrit un service Web en « **boîte noire** » : ce qu'il fait et comment l'appeler.

- Quelles opérations, entrées, sorties, types ?
- Comment y accéder : adresse, protocole ?
- Quel format pour les messages requête / réponse ?

```
<definitions>
  <types/>
  <message/>
  <portType/>      Description abstraite
                   CE QUE fait le service : opérations, paramètres, messages
  <binding/>
  <service/>       Description concrète
                   COMMENT y accéder : protocole et URI du service
</definitions>
```

Réutilisabilité : une même partie abstraite peut être exposée par **plusieurs parties concrètes** (ex. binding HTTP, binding SMTP) : les opérations ne sont écrites qu'une fois.

WSDL 1.1 (Note W3C, 2001) est la version la plus utilisée ; WSDL 2.0 est une Recommandation W3C.

12 LES 7 ÉLÉMENTS WSDL

Abstraite | Liaison | Concrète

— exemple : GestionEtudiantService

Élément	Rôle	Exemple
`types`	Types de données (schéma XSD)	getEtudiantRequest, getEtudiantResponse
`message`	Messages échangés (entrée, sortie)	getEtudiantInput, getEtudiantOutput
`operation`	Action offerte, avec ses messages	getEtudiant
`portType`	Ensemble d'opérations : l'interface	GestionEtudiantPortType
`binding`	Protocole et format concrets d'un portType	GestionEtudiantBinding (SOAP sur HTTP)
`port`	Adresse d'accès (endpoint) d'un binding	.../services/gestionEtudiant
`service`	Ensemble de ports	GestionEtudiantService

Lecture d'un WSDL : types → message → portType (ce que fait le service), puis binding → service (comment l'appeler et où).

13 DEUX APPROCHES DE DÉVELOPPEMENT

Bottom-up (code d'abord)

1 Implémenter la classe métier
2 Compiler, déployer, tester
3 Le WSDL est **généré** à partir du code

Top-down (contrat d'abord)

1 Générer classes et squelette **depuis le WSDL**
2 Compléter le squelette
3 Compiler, déployer, tester

14 L'ANNUAIRE UDDI

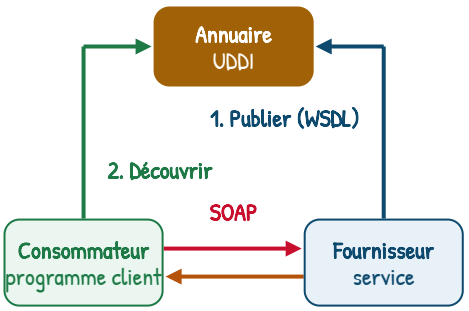
UDDI (Universal Description, Discovery and Integration) : registre basé sur XML, indépendant de la plateforme, où des entreprises référencent leurs services Web. C'est la **couche de découverte**.

Spécification technique : établir un annuaire distribué des entreprises et de leurs services Web.

Registre (UDDI registry) : chacun peut y rechercher ; toute entreprise peut s'y enregistrer avec ses services.

Historique : le registre public UDDI a fermé en 2006 ; on utilise aujourd'hui des registres internes ou des portails d'API.

15 FONCTIONNEMENT GLOBAL



1 Le fournisseur **publie** la description (WSDL) dans l'annuaire.

2 Le consommateur **découvre** le service et récupère son WSDL.

3 Il **appelle** le service : requête SOAP.

4 Le service renvoie la **réponse SOAP** (ou un Fault).

Transport : HTTP / HTTPS.

POINTS À RETENIR

Annoter un message SOAP : Envelope, Header, Body, Fault ; vocabulaire soap: et métier.

Lire un WSDL : repérer la partie abstraite (types, message, portType) et la partie concrète (binding, service).

Distinguer : bien formé / valide, bottom-up / top-down, SOAP / HTTP / WSDL / UDDI.